MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

# Mini Project Report
of
# Database Systems Lab (CSS 2212)

# Role Based Course Management and Assessment System

**SUBMITTED**
**BY**


| STUDENT NAME | REG. NO | ROLL NO | SECTIONS |
|---|---|---|---|
| Aditya Jain | 240962322 | 20 | AIML C1 |
| Saihaj Singh | 240962332 | 21 | AIML C1 |
| Agastya Bhardwaj | 240962340 | 22 | AIML C1 |

School of Computer Engineering
Manipal Institute of Technology, Manipal.

April 2026

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
*(A constituent unit of MAHE, Manipal)*

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

**Manipal**
**30/03/2026**

# CERTIFICATE

This is to certify that the project titled **Role-Based Course Management and Assessment System** is a record of the bonafide work done by **Students: Aditya Jain (Reg. No. 240962322), Saihaj Singh (Reg. No. 240962332), and Agastya Bhardwaj (Reg. No. 240962340)** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in Computer Science (Artificial Intelligence and Machine Learning) ENGINEERING of Manipal Institute of Technology, Manipal, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

## Name and Signature of Examiners:

1. **Dr. Sheena K S, Assistant Professor, SCE, MIT Manipal.**

2. **Dr. Anoop B N, Assistant Professor, SCE. MIT Manipal.**

## TABLE OF CONTENTS


| | Topic | Page No. |
|---|---|---|
| — | Abstract | 4 |
| 1 | Introduction | 5 |
| 2 | Problem Statement & Objectives | 7 |
| 3 | Methodology | 9 |
| 4 | ER Diagram & Relational Schema with Sample Data | 11 |
| 5 | DDL Commands & PL/SQL Procedures / Functions / Triggers | 18 |
| 6 | Results & Snapshots | 31 |
| 7 | Conclusion, Limitations & Future Work | 37 |
| 8 | References | 39 |

# ABSTRACT

Educational institutions deal with a large volume of academic data including student enrollments, course materials, assignment submissions, grading, and feedback. Managing these activities manually or through fragmented software tools leads to inefficiencies such as missed deadlines, inconsistent grading, lack of transparency, and poor communication between students and instructors.

This project presents the design and implementation of a Role-Based Course Management and Assessment System (RCMAS) — a full-stack web application backed by a relational database. The system is designed to serve multiple user roles, namely students, instructors, teaching assistants (TAs), and administrators, each with clearly defined permissions and views.

The backend is built using Python (Flask) with a MySQL relational database. The frontend is a single-page HTML/CSS/JavaScript application. The system supports secure JWT-based authentication, bcrypt password hashing, and a connection pooling mechanism for efficient database interactions.

Key features include structured course creation and enrollment, assignment lifecycle management (creation, submission, late detection, grading), automatic grade letter computation via database triggers, discussion threads per assignment, course-level polls, audit logging, and comprehensive reporting views. The database design employs 13 normalized tables, 10 stored procedures, 10 triggers, and 8 views, demonstrating the full spectrum of relational database capabilities in a real-world academic context.

# CHAPTER 1: INTRODUCTION

Academic institutions require robust, role-aware systems to manage the full lifecycle of course delivery from enrollment and assignment creation through grading, discussion, and feedback. The Role-Based Course Management and Assessment System (RCMAS) addresses this need by providing a structured, scalable relational database backend that enforces role-based access control and automates key academic workflows.

This project implements a complete database-driven system covering student enrollment, assignment management, submission and grading, discussion threads, course-level polls, and a comprehensive audit trail. The system is built on MySQL and connects to a Python Flask backend via a connection pool, providing a single-page web application as the user interface.

The project applies core DBMS principles including entity-relationship modelling, relational schema design, normalization, and advanced SQL features such as stored procedures, triggers, and views to automate business logic at the database level.

## 1.1 System Architecture

The application follows a three-tier architecture: an HTML/CSS/JavaScript presentation layer, a Flask-based application logic layer, and a MySQL data layer. JWT tokens carry all session state, eliminating server-side session storage. The diagram below illustrates the interaction between all three tiers.

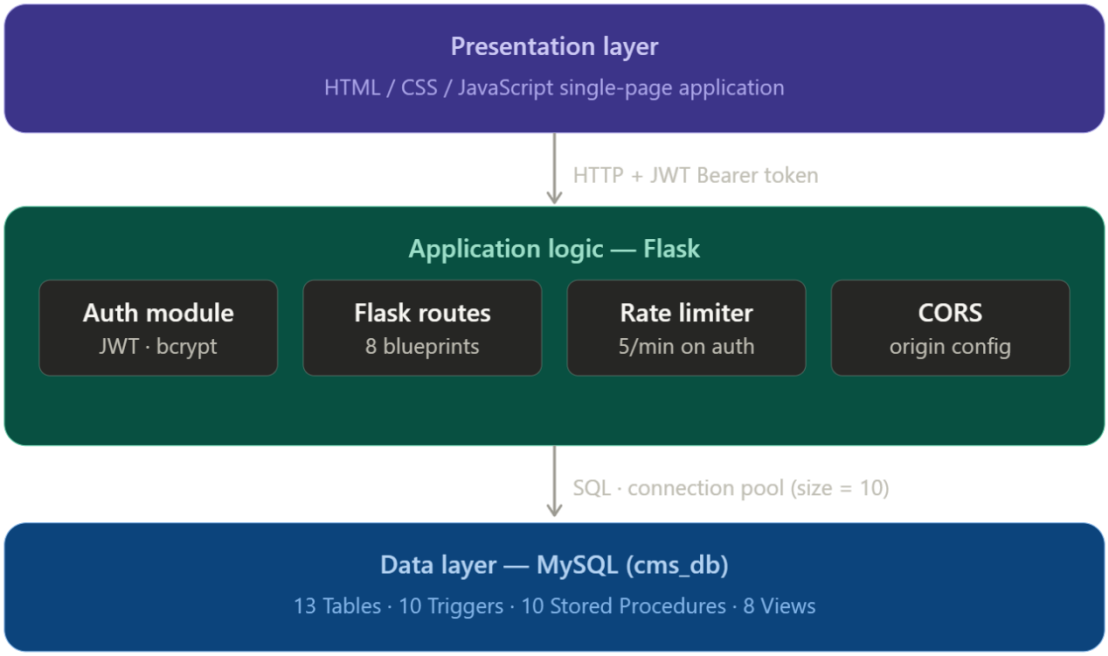



*Figure 1.1 Three-tier system architecture: HTML/JS → Flask → MySQL*

The Flask application is organized into 8 Blueprints, each responsible for a logical domain (auth, courses, assignments, submissions, grades, discussions, polls, admin). Rate limiting is applied via flask_limiter 5 requests/minute on auth endpoints and 200 requests/day globally.

## 1.2 Scope of the Project

The scope of this project includes the design and implementation of a normalized relational database schema with 13 entities, implementation of all CRUD operations, multi-table JOIN queries, window-function analytics, 10 stored procedures, 10 triggers, and 8 pre-built views. A single-page web application demonstrates live database connectivity with role-adaptive UI panels for all four user roles.

## 1.3 Tools and Technologies

- Database: MySQL 8.0 (utf8mb4 charset, utf8mb4_unicode_ci collation)
- Query Language: SQL / PL/SQL (Stored Procedures, Triggers, Views)
- Backend Framework: Python 3 with Flask (microframework)
- Authentication: JSON Web Tokens (PyJWT), bcrypt password hashing
- DB Connector: mysql-connector-python with MySQLConnectionPool (pool size = 10)
- Frontend: Single-page HTML5 / CSS3 / Vanilla JavaScript application
- ER Modelling Tool: MySQL Workbench / draw.io
- Other Libraries: flask-cors, flask-limiter, python-dotenv

# CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

## 2.1 Problem Statement

Educational platforms require efficient management of users across multiple roles, course structures, assignment workflows, grading pipelines, and communication channels. In the absence of a structured database system, issues such as duplicate enrollments, over-capacity courses, inconsistent grade calculations, missed submission deadlines, and lack of audit trails may arise.

There is a need for a structured relational database system that can:

- Store and manage multi-role user accounts securely with role-based access control.
- Maintain course and department structures with capacity enforcement.
- Handle the full assignment lifecycle creation, submission, late detection, grading, and grade history.
- Automatically compute and update grade letters and final course grades via database triggers.

- Support threaded assignment discussions and course-level polls.
- Maintain a comprehensive audit log of all significant data changes.
- Provide pre-built analytical views for grade reports, enrollment summaries, and pending grading queues.

## 2.2 Functional Requirements

FR1: User Registration and Authentication The system must allow users to register with role designation and log in securely. Passwords are hashed using bcrypt. Authentication is token-based using JSON Web Tokens (JWT), which expire after a configurable period.

FR2: Role-Based Access Control Each endpoint and operation must enforce role checks. Students may enroll in courses and submit assignments. Instructors and TAs may create assignments and grade submissions. Only instructors and admins may create courses. Admins have full access including audit log visibility.

FR3: Course Management Instructors must be able to create courses with department linkage, capacity limits, and semester designation. Students must be able to enroll in active courses within capacity. The system must prevent an instructor from enrolling in their own course.

FR4: Assignment Management Instructors and TAs must be able to create assignments with configurable due dates, marks, and late submission policies. The system must automatically reject submissions past the deadline if late submissions are disallowed, or mark them as late if they are allowed.

FR5: Submission and Grading Students must be able to submit assignments while enrolled. Instructors and TAs must be able to grade submissions with marks and feedback. Upon grading, the grade letter must be automatically computed. The student's final course grade must be automatically recalculated.

FR6: Discussion Threads All users must be able to post messages in discussion threads associated with assignments. Replies must be tracked via parent-child relationships. Instructors may pin important messages.

FR7: Polls Instructors must be able to create polls with up to four options. Students may vote once per poll. Expired polls must reject new votes. Poll results must be viewable as vote counts and percentages.

FR8: Reporting and Analytics The system must provide pre-built views for student grade reports, course enrollment summaries, pending grading queues, assignment statistics, poll results, and a human-readable audit log.

FR9: Audit Logging All significant actions (registrations, course creation, enrollments, submissions, discussions) must be automatically logged to the Audit_Log table with actor information and timestamps, without requiring application-level code for each action.

### 2.3 Objectives

- Design a normalized relational database schema up to Third Normal Form (3NF) for managing academic operations.
- Implement all 13 tables with appropriate Primary Key, Foreign Key, NOT NULL, UNIQUE, and CHECK constraints.
- Develop basic SQL queries (SELECT, INSERT, UPDATE, DELETE) for all entities.
- Write complex queries using JOIN, GROUP BY, aggregate functions, subqueries, and window functions.
- Implement 10 stored procedures encapsulating core business logic.
- Implement 10 triggers enforcing business rules automatically at the database level.
- Define 8 views for reporting and analytics.
- Develop a role-adaptive single-page web application demonstrating live database connectivity.

# CHAPTER 3: METHODOLOGY

The project follows the standard Database Development Life Cycle (DDLC) consisting of the following phases.

## 3.1 Requirement Gathering

System requirements were identified by analysing the functional needs of an academic course management platform. Entities, attributes, relationships, and business rules were documented based on common academic workflows including user management, course delivery, assignment grading, and student communication.

## 3.2 Conceptual Design

An Entity-Relationship (ER) Diagram was created to represent 13 primary entities: Users, Departments, Courses, Course_TA, Enrollments, Assignments, Submissions, Grade_History, Discussions, Polls, Poll_Options, Poll_Responses, and Audit_Log. Relationships between entities were defined along with their cardinalities — for example, one Instructor teaches many Courses (1:N), and students enroll in many Courses through the Enrollments junction table (M:N resolved).

## 3.3 Logical Design & Normalization

The ER model was converted into a relational schema. Normalization was applied up to Third Normal Form (3NF):

- 1NF: All attributes contain atomic values; no repeating groups. The role column uses an ENUM type to restrict values to a defined domain.

- 2NF: All non-key attributes are fully dependent on the entire primary key. Composite keys in Course_TA, Poll_Responses, and the unique constraints in Enrollments and Submissions were verified for full functional dependency.
- 3NF: No transitive dependencies exist — all non-key attributes depend only on their primary key. For example, department details are stored in the Departments table, not repeated in Courses.

### 3.4 Physical Design

Tables were created in MySQL with all necessary constraints including Primary Keys, Foreign Keys, NOT NULL, UNIQUE (e.g., uq_enroll on student-course pairs, uq_submission on assignment-student pairs), and CHECK constraints (e.g., credits BETWEEN 1 AND 6, final_grade BETWEEN 0 AND 100). ON DELETE behaviours were carefully chosen — CASCADE for dependent child records, RESTRICT to preserve accountability, and SET NULL to retain orphaned audit records.

### 3.5 Implementation

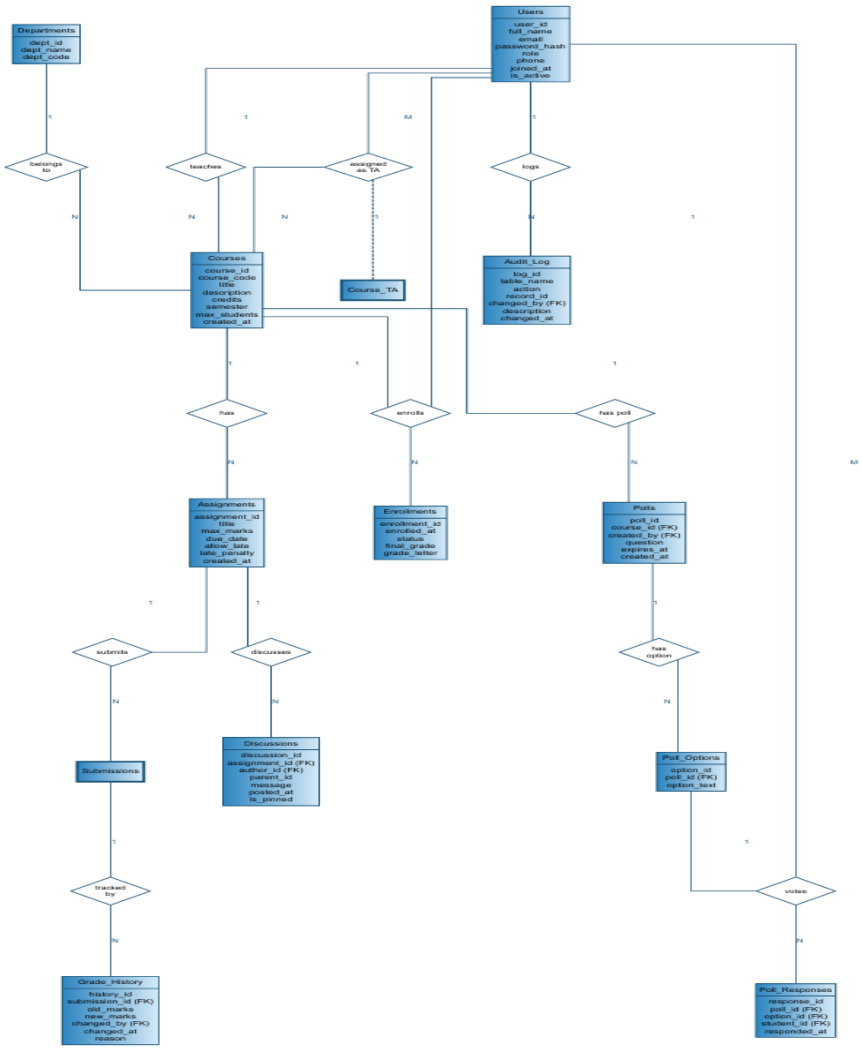
Sample data was inserted into all 13 tables covering 4 departments, 14 users across 4 roles, 4 courses, 10 assignments, 11 submissions with grades, discussion threads, and poll responses. SQL queries of increasing complexity were implemented from basic CRUD through multi-table JOINs, aggregations, subqueries, and window functions followed by 10 stored procedures, 10 triggers, and 8 views. A Flask web application was developed to provide live database connectivity with a role-adaptive single-page UI.

### 3.6 Testing & Validation

All constraints were tested by attempting invalid insertions. Trigger behaviour was verified  for example, submitting past a deadline on a non-late assignment raises SQLSTATE 45000, and grading a submission automatically updates the enrollment's final grade. Stored procedure output was validated against manual calculations. JWT expiry and role-based route protection were tested across all four user roles.

# CHAPTER 4: ER DIAGRAM & RELATIONAL SCHEMA WITH SAMPLE DATA

## 4.1 Relational Schema



**Key relationships:**

- Users (1) ⟶ Courses (N): One instructor teaches many courses.
- Departments (1) ⟶ Courses (N): One department contains many courses.
- Users (1) ⟶ Enrollments (N): One student can have many enrollments.
- Courses (1) ⟶ Enrollments (N): One course can have many enrolled students.
- Courses (1) ⟶ Course_TA (N): One course can have many teaching assistants assigned.
- Users (1) ⟶ Course_TA (N): One TA can be assigned to many courses.
- Courses (1) ⟶ Assignments (N): One course contains many assignments.
- Users (1) ⟶ Assignments (N): One instructor or TA can create many assignments.
- Assignments (1) ⟶ Submissions (N): One assignment receives many submissions.
- Users (1) ⟶ Submissions (N): One student can submit many assignment responses.
- Users (1) ⟶ Submissions (N): One grader can grade many submissions.
- Submissions (1) ⟶ Grade_History (N): One submission can have many grade history records.
- Assignments (1) ⟶ Discussions (N): One assignment contains many discussion messages.
- Users (1) ⟶ Discussions (N): One user can post many discussion messages.
- Discussions (1) ⟶ Discussions (N): One discussion message can have many replies (self-referencing relationship).
- Courses (1) ⟶ Polls (N): One course can have many polls.
- Polls (1) ⟶ Poll_Options (N): One poll contains many options.

- Polls (1) ⟶ Poll_Responses (N): One poll can receive many student responses.
- Users (1) ⟶ Poll_Responses (N): One student can respond to many polls.
- Users (1) ⟶ Audit_Log (N): One user can generate many audit log records.

### 4.2 Relational Schema

The following relational tables were derived from the ER diagram:

**<u>Users</u>**
( user_id, full_name, email*, password_hash, role, phone, joined_at, is_active )

**<u>Departments</u>**
( dept_id, dept_name*, dept_code* )

**<u>Courses</u>**
( course_id, course_code*, title, description, dept_id#, instructor_id#, credits, max_students, semester, is_active, created_at )

**<u>Course_TA</u>**
( course_id#, ta_id#, assigned_at)
[Composite PK]

**<u>Enrollments</u>**
( enrollment_id, student_id#, course_id#, enrolled_at, status, final_grade, grade_letter)
— UNIQUE(student_id, course_id)

**Assignments**
( assignment_id, course_id#, created_by#, title, description, max_marks, due_date, allow_late, late_penalty, created_at )

**Submissions**
( submission_id, assignment_id#, student_id#, submitted_at, file_path, content_text, is_late, marks_obtained, grade_letter, graded_by#, graded_at, feedback, status )
— UNIQUE(assignment_id, student_id)

**Grade_History**
( history_id, submission_id#, old_marks, new_marks, changed_by#, changed_at, reason )

**Discussions**
( discussion_id, assignment_id#, author_id#, parent_id#, message, posted_at, is_pinned )

**Polls**
( poll_id, course_id#, created_by#, question, expires_at, created_at )

**Poll_Options**
( option_id, poll_id#, option_text )

**Poll_Responses**
( response_id, poll_id#, option_id#, student_id#, responded_at )
— UNIQUE(poll_id, student_id)

**Audit_Log**
( log_id, table_name, action, record_id, changed_by#, description, changed_at )

*( = UNIQUE constraint, # = Foreign Key)**

### 4.3 Sample Data

## Users Table (Sample)

| user_id | full_name | email | role | phone |
|---|---|---|---|---|
| 1 | Admin User | admin@cms.edu | admin | 9000000000 |
| 2 | Dr. Priya Sharma | priya.sharma@cms.edu | instructor | 9111111111 |
| 5 | Ravi Kumar | ravi.kumar@cms.edu | ta | 9444444444 |
| 7 | Aditya Verma | aditya.v@student.cms.edu | student | 8001111111 |
| 8 | Meera Nair | meera.n@student.cms.edu | student | 8002222222 |

## Courses Table (Sample)

| course_id | course_code | title | dept_id | instructor_id | credits | max_students | semester |
|---|---|---|---|---|---|---|---|
| 1 | CS301 | Database Management Systems | 1 | 2 | 4 | 5 | Spring 2025 |
| 2 | CS401 | Machine Learning | 1 | 2 | 4 | 5 | Spring 2025 |
| 3 | MATH201 | Discrete Mathematics | 2 | 3 | 3 | 5 | Spring 2025 |

## Enrollments Table (Sample)

| enrollment_id | student_id | course_id | status | final_grade | grade_letter |
|---|---|---|---|---|---|
| 1 | 7 | 1 | active | 88.00 | A |
| 2 | 8 | 1 | active | 76.50 | B |
| 3 | 9 | 1 | active | 72.00 | B |

## Submissions Table (Sample)

| submission_id | assignment_id | student_id | is_late | marks_obtained | grade_letter | status |
|---|---|---|---|---|---|---|

| submission_id | assignment_id | student_id | is_late | marks_obtained | grade_letter | status |
|---|---|---|---|---|---|---|
| 1 | 1 | 7 | FALSE | 22.00 | A | graded |
| 2 | 1 | 8 | FALSE | 20.00 | A | graded |
| 3 | 1 | 9 | FALSE | 18.00 | B | graded |

# CHAPTER 5: DDL COMMANDS & PL/SQL PROCEDURES / FUNCTIONS / TRIGGERS

## 5.1 Database Creation & DDL

**Creating the database:**

```
CREATE DATABASE cms_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE cms_db;
```

The database uses utf8mb4 character encoding, which supports the full Unicode character set. The utf8mb4_unicode_ci collation provides case-insensitive, language-aware string comparison.

**Table 1: Users**

```
CREATE TABLE Users (
   user_id       INT           AUTO_INCREMENT PRIMARY KEY,
   full_name     VARCHAR(120)  NOT NULL,
   email         VARCHAR(180)  NOT NULL UNIQUE,
   password_hash VARCHAR(255)  NOT NULL,
   role          ENUM('student','instructor','ta','admin')  NOT  NULL DEFAULT 'student',
   phone         VARCHAR(20),
   joined_at     DATETIME      NOT  NULL  DEFAULT CURRENT_TIMESTAMP,
   is_active     BOOLEAN       NOT NULL DEFAULT TRUE,
   CONSTRAINT chk_email CHECK (email LIKE '%@%.%')
);
```

The Users table is the central identity table. The role ENUM restricts values to valid roles only. The chk_email CHECK constraint enforces basic email format. Passwords are stored as bcrypt hashes, never as plain text.

**Table 2: Departments**

```
CREATE TABLE Departments (
   dept_id   INT         AUTO_INCREMENT PRIMARY KEY,
   dept_name VARCHAR(100) NOT NULL UNIQUE,
   dept_code VARCHAR(10)  NOT NULL UNIQUE
);
```

**Table 3: Courses**

```
CREATE TABLE Courses (
   course_id     INT          AUTO_INCREMENT PRIMARY KEY,
   course_code   VARCHAR(20)   NOT NULL UNIQUE,
   title         VARCHAR(200)  NOT NULL,
   description   TEXT,
   dept_id       INT          NOT NULL,
   instructor_id INT          NOT NULL,
   credits       TINYINT      NOT NULL DEFAULT 3,
   max_students  INT          NOT NULL DEFAULT 60,
   semester      VARCHAR(20)   NOT NULL,
   is_active     BOOLEAN      NOT NULL DEFAULT TRUE,
   created_at            DATETIME                NOT   NULL   DEFAULT
CURRENT_TIMESTAMP,
   CONSTRAINT   fk_course_dept              FOREIGN   KEY   (dept_id)
REFERENCES Departments(dept_id)  ON DELETE RESTRICT,
   CONSTRAINT  fk_course_instructor  FOREIGN  KEY  (instructor_id)
REFERENCES Users(user_id)        ON DELETE RESTRICT,
   CONSTRAINT  chk_credits                CHECK  (credits  BETWEEN  1
AND 6),
   CONSTRAINT chk_max_students     CHECK (max_students > 0)
);
```

ON DELETE RESTRICT on both foreign keys prevents accidental deletion of departments or instructors that still have active courses.

**Table 5: Enrollments**

```
CREATE TABLE Enrollments (
   enrollment_id INT      AUTO_INCREMENT PRIMARY KEY,
   student_id    INT      NOT NULL,
   course_id     INT      NOT NULL,
   enrolled_at              DATETIME      NOT      NULL      DEFAULT
CURRENT_TIMESTAMP,
   status                 ENUM('active','dropped','completed')   NOT   NULL
DEFAULT 'active',
   final_grade   DECIMAL(5,2),
   grade_letter  CHAR(2),
   CONSTRAINT    fk_enroll_student    FOREIGN    KEY    (student_id)
REFERENCES Users(user_id)     ON DELETE CASCADE,
   CONSTRAINT    fk_enroll_course      FOREIGN    KEY    (course_id)
REFERENCES Courses(course_id) ON DELETE CASCADE,
   CONSTRAINT uq_enroll         UNIQUE (student_id, course_id),
   CONSTRAINT chk_final_grade    CHECK (final_grade IS NULL OR
final_grade BETWEEN 0 AND 100)
);
```

The uq_enroll UNIQUE constraint prevents duplicate enrollments. The final_grade and grade_letter are computed automatically by Trigger 7 whenever a submission is graded.

**<u>Table 7: Submissions</u>**

```
CREATE TABLE Submissions (
   submission_id   INT          AUTO_INCREMENT PRIMARY KEY,
   assignment_id   INT          NOT NULL,
   student_id      INT          NOT NULL,
   submitted_at          DATETIME                NOT   NULL   DEFAULT
CURRENT_TIMESTAMP,
   file_path       VARCHAR(500),
   content_text    TEXT,
   is_late         BOOLEAN       NOT NULL DEFAULT FALSE,
   marks_obtained  DECIMAL(6,2),
```

```
    grade_letter    CHAR(2),
    graded_by       INT,
    graded_at       DATETIME,
    feedback        TEXT,
    status          ENUM('submitted','graded','returned') NOT NULL
DEFAULT 'submitted',
    CONSTRAINT fk_sub_assignment FOREIGN KEY (assignment_id)
REFERENCES Assignments(assignment_id) ON DELETE CASCADE,
    CONSTRAINT fk_sub_student FOREIGN KEY (student_id)
REFERENCES Users(user_id) ON DELETE CASCADE,
    CONSTRAINT fk_sub_grader FOREIGN KEY (graded_by)
REFERENCES Users(user_id) ON DELETE SET NULL,
    CONSTRAINT uq_submission UNIQUE (assignment_id,
student_id),
    CONSTRAINT chk_marks CHECK (marks_obtained IS NULL
OR marks_obtained >= 0)
);
```

ON DELETE SET NULL on graded_by ensures that deleting a grader user does not cascade-delete grade records. The grade_letter is auto-populated by a BEFORE UPDATE trigger.

## 5.2 Basic DML Queries

**List all users with their roles:**

```
SELECT user_id, full_name, email, role, joined_at
FROM Users
ORDER BY role, full_name;
```

**List all active courses with instructor name:**

```
SELECT c.course_code, c.title, c.semester, c.credits,
       c.max_students, u.full_name AS instructor
FROM Courses c
JOIN Users u ON u.user_id = c.instructor_id
```

```
WHERE c.is_active = TRUE;
```

**List all students enrolled in DBMS (course_id = 1):**

```
SELECT u.user_id, u.full_name, u.email, e.enrolled_at, e.status
FROM Enrollments e
JOIN Users u ON u.user_id = e.student_id
WHERE e.course_id = 1 AND e.status = 'active';
```

**Update a student's email address:**

```
UPDATE Users
SET email = 'aditya.verma@student.cms.edu'
WHERE user_id = 7;
```

**5.3 Complex Queries**

**All submissions for Assignment 1 with student names and grader names (multi-table JOIN):**

```
SELECT
    s.submission_id,
    u.full_name          AS student_name,
    s.submitted_at,
    s.is_late,
    s.marks_obtained,
    a.max_marks,
    s.grade_letter,
    s.status,
    g.full_name          AS graded_by
FROM Submissions s
JOIN Users       u ON u.user_id       = s.student_id
JOIN Assignments a ON a.assignment_id = s.assignment_id
LEFT JOIN Users  g ON g.user_id       = s.graded_by
WHERE s.assignment_id = 1
ORDER BY s.marks_obtained DESC;
```

**Students who scored above the class average for Assignment 1 (subquery):**

```
SELECT u.full_name, s.marks_obtained
FROM Submissions s
JOIN Users u ON u.user_id = s.student_id
WHERE s.assignment_id = 1
  AND s.marks_obtained > (
      SELECT AVG(marks_obtained)
      FROM Submissions WHERE assignment_id = 1
  )
ORDER BY s.marks_obtained DESC;
```

**Poll results with vote counts and window-function percentages:**

```
SELECT
    p.poll_id, p.question, po.option_text,
    COUNT(pr.response_id) AS votes,
    ROUND(COUNT(pr.response_id) * 100.0 /
        SUM(COUNT(pr.response_id)) OVER (PARTITION BY
p.poll_id), 1) AS pct
FROM Polls p
JOIN Poll_Options po ON po.poll_id = p.poll_id
LEFT JOIN Poll_Responses pr ON pr.option_id = po.option_id
GROUP BY p.poll_id, p.question, po.option_id, po.option_text
ORDER BY p.poll_id, votes DESC;
```

**Rank students by marks within each assignment (window function):**

```
SELECT
    a.title AS assignment,
    u.full_name AS student,
    s.marks_obtained,
RANK() OVER (PARTITION BY s.assignment_id ORDER BY
s.marks_obtained DESC) AS `rank`
```

```
FROM Submissions s
JOIN Users       u ON u.user_id       = s.student_id
JOIN Assignments a ON a.assignment_id = s.assignment_id
WHERE s.marks_obtained IS NOT NULL;
```

**Top-scoring student in each course (correlated subquery):**

```
SELECT c.course_code, u.full_name AS top_student, e.final_grade
FROM Enrollments e
JOIN Courses c ON c.course_id = e.course_id
JOIN Users   u ON u.user_id   = e.student_id
WHERE e.final_grade = (
   SELECT MAX(e2.final_grade)
   FROM Enrollments e2
   WHERE e2.course_id = e.course_id
)
AND e.final_grade IS NOT NULL;
```

### 5.4 Stored Procedures

<u>SP1: sp_register_user</u>
Registers a new user with role validation and audit logging. Validates that p_role is one of the four allowed values, inserts the user record, captures the auto-generated ID via LAST_INSERT_ID(), and inserts an audit log entry. Uses an EXIT HANDLER for SQLEXCEPTION to roll back on failure.

```
CALL sp_register_user('Jane Doe', 'jane@student.cms.edu',
   SHA2('pass123',256), 'student', '9876543210', @uid, @msg);
SELECT @uid, @msg;
```

<u>SP2: sp_enroll_student</u>
Enrolls a student in a course after verifying role, checking for duplicate enrollment, and allowing the trg_enrollment_cap trigger to validate capacity.

SP3: sp_create_course
Creates a course after verifying the caller is an instructor or admin. The trg_log_course_created trigger fires automatically to record the creation in Audit_Log.

SP4: sp_create_assignment
Creates an assignment after verifying the creator's role and confirming the due date is in the future (p_due_date > NOW()).

SP5: sp_submit_assignment
Records a student submission after verifying active enrollment. The trg_submission_late_check trigger fires BEFORE INSERT to set is_late or reject the submission. Uses GET DIAGNOSTICS CONDITION 1 to surface trigger SIGNAL messages back to the caller.

SP6: sp_grade_submission
Records a grade after role verification and bounds checking against max_marks. The trg_auto_grade_letter, trg_record_grade_history, and trg_recalc_final_grade triggers all fire automatically on the UPDATE.

CALL sp_grade_submission(6, 2, 47.0, 'Clean code, add comments next time.', @gmsg);
SELECT @gmsg;

SP7: sp_post_discussion
Posts a new discussion message or reply. Uses NULLIF(p_parent_id, 0) to convert a zero value to NULL for top-level threads.

SP8: sp_create_poll
Creates a poll with up to four options in a single transaction. Conditionally inserts each option only if the parameter is NOT NULL.

SP9: sp_respond_poll

Records a student's vote after checking poll expiry. The UNIQUE(poll_id, student_id) constraint on Poll_Responses prevents double voting at the database level.

SP10: sp_drop_student
Changes a student's enrollment status from active to dropped. Uses ROW_COUNT() to detect whether a record was actually changed and returns an informative message if no active enrollment was found.

### 5.5 Triggers

Trigger 1: trg_submission_late_check (BEFORE INSERT on Submissions)
Automatically determines whether a submission is late. Fetches due_date and allow_late from Assignments, then either rejects the insert with a SIGNAL, marks it late, or marks it on-time. This enforces the business rule entirely at the database level.

```
IF NOW() > v_due_date THEN
   IF v_allow_late = FALSE THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Submission rejected: deadline has passed...';
   ELSE
      SET NEW.is_late = TRUE;
   END IF;
ELSE
   SET NEW.is_late = FALSE;
END IF;
```

Trigger 2: trg_log_new_submission (AFTER INSERT on Submissions)
Creates an audit trail entry every time a submission is recorded, using NEW.submission_id available after INSERT.

Trigger 3: trg_auto_grade_letter (BEFORE UPDATE on Submissions)
Automatically computes and sets grade_letter whenever marks_obtained is assigned or changed. Only activates when marks_obtained actually changes (guarded by an IF comparing OLD and NEW values).

```
SET NEW.grade_letter = CASE
   WHEN v_pct >= 90 THEN 'A+'
   WHEN v_pct >= 80 THEN 'A'
   WHEN v_pct >= 70 THEN 'B'
   WHEN v_pct >= 60 THEN 'C'
   WHEN v_pct >= 50 THEN 'D'
   ELSE 'F'
END;
SET NEW.status = 'graded';
```

Trigger 4: trg_record_grade_history (AFTER UPDATE on Submissions)
Logs every grade change to Grade_History for academic audit and integrity, including old marks, new marks, grader ID, and timestamp.

Trigger 5: trg_enrollment_cap (BEFORE INSERT on Enrollments)
Prevents over-enrollment by checking the current active enrollment count against the course maximum. Uses two separate SELECT statements (one for course metadata, one for the COUNT aggregate) to avoid ONLY_FULL_GROUP_BY errors. Raises appropriate errors for inactive courses or full capacity.

Trigger 6: trg_log_enrollment (AFTER INSERT on Enrollments) Writes an audit log entry documenting every new student enrollment.

Trigger 7: trg_recalc_final_grade (AFTER UPDATE on Submissions)
Automatically recalculates the student's cumulative final grade in Enrollments whenever a submission grade changes. Computes the average of (marks_obtained / max_marks) * 100 across all graded submissions for that student in that course and updates final_grade and grade_letter in Enrollments. This creates a live-updating final grade without any application-level computation.

Trigger 8: trg_no_self_enroll_as_instructor (BEFORE INSERT on Enrollments)
Prevents an instructor from enrolling as a student in their own course. Fetches the course's instructor_id and compares it to NEW.student_id.

Trigger 9: trg_log_discussion (AFTER INSERT on Discussions) Logs every new discussion post or reply, distinguishing between new threads and replies using IF(NEW.parent_id IS NULL, ' (new thread)', CONCAT(' (reply to ', NEW.parent_id, ')')).

Trigger 10: trg_log_course_created (AFTER INSERT on Courses) Automatically records every new course creation in the audit log with the course code and title.

**5.6 Views**

View 1: vw_student_grades
Per-assignment grade report for every student. Joins Users, Enrollments, Courses, Assignments, and Submissions using LEFT JOIN to include assignments not yet submitted. Used for the student grade dashboard.

View 2: vw_course_summary
Aggregated course statistics including enrolled student count, fill percentage, number of assignments, and average final grade. Used by the admin/instructor dashboard.

View 3: vw_pending_submissions
Lists all submissions with status = 'submitted' awaiting grading. Includes days overdue via DATEDIFF(NOW(), due_date).

View 4: vw_assignment_stats
Per-assignment statistics: total submissions, late submission count, average marks, min and max marks. Renamed aggregate aliases

(min_marks_obtained, max_marks_obtained) avoid column name collision with max_marks.

View 5: vw_student_enrollment_detail
Each student's enrolled courses with current final grade, grade letter, enrollment date, and instructor name.

View 6: vw_discussion_threads
Flat view of all discussion posts with author name, role, and threading order, sorted by pinned status, thread group, and post time.

View 7: vw_poll_results
Poll vote counts and percentages per option. Uses a correlated subquery within NULLIF to safely handle polls with zero total votes.

View 8: vw_audit_log_detail
Human-readable audit log joining the changed_by user's full name. Ordered by changed_at DESC for most-recent-first display.

**Sample view query:**

```
-- Student grades for student Aditya (user_id = 7)
SELECT * FROM vw_student_grades WHERE student_id = 7;

-- Course summary
SELECT * FROM vw_course_summary;

-- Pending grading queue
SELECT * FROM vw_pending_submissions;
```

# CHAPTER 6: RESULTS & SNAPSHOTS

This chapter presents the output of key queries and demonstrates the behaviour of triggers, stored procedures, and views. Screenshots from MySQL Workbench and the EduCore web application are included for each section.

## 6.1 Table Creation Output

```
1 •  SHOW TABLES;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Tables_in_cms_db |
|---|
| assignments |
| audit_log |
| course_ta |
| courses |
| departments |
| discussions |
| enrollments |
| grade_history |
| poll_options |
| poll_responses |
| polls |
| submissions |
| users |

## 6.2 Basic Query Output

```sql
SELECT user_id, full_name, email, role, joined_at
FROM Users
ORDER BY role, full_name;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

| user_id | full_name | email | role | joined_at |
|---|---|---|---|---|
| 7 | Aditya Verma | aditya.v@student.cms.edu | student | 2026-03-31 13:23:37 |
| 13 | Aryan Kapoor | aryan.k@student.cms.edu | student | 2026-03-31 13:23:37 |
| 10 | Kavya Patel | kavya.p@student.cms.edu | student | 2026-03-31 13:23:37 |
| 8 | Meera Nair | meera.n@student.cms.edu | student | 2026-03-31 13:23:37 |
| 12 | Pooja Iyer | pooja.i@student.cms.edu | student | 2026-03-31 13:23:37 |
| 9 | Rohan Singh | rohan.s@student.cms.edu | student | 2026-03-31 13:23:37 |
| 14 | Shreya Kulkarni | shreya.k@student.cms.edu | student | 2026-03-31 13:23:37 |
| 11 | Siddharth Jain | sidd.j@student.cms.edu | student | 2026-03-31 13:23:37 |
| 2 | Dr. Priya Sharma | priya.sharma@cms.edu | instructor | 2026-03-31 13:23:37 |
| 4 | Dr. Sunita Rao | sunita.rao@cms.edu | instructor | 2026-03-31 13:23:37 |
| 3 | Prof. Arjun Mehta | arjun.mehta@cms.edu | instructor | 2026-03-31 13:23:37 |
| 6 | Anita Desai | anita.desai@cms.edu | ta | 2026-03-31 13:23:37 |
| 5 | Ravi Kumar | ravi.kumar@cms.edu | ta | 2026-03-31 13:23:37 |
| 1 | Admin User | admin@cms.edu | admin | 2026-03-31 13:23:37 |
| NULL | NULL | NULL | NULL | NULL |

```sql
SELECT c.course_code, c.title, c.semester, c.credits,
       c.max_students, u.full_name AS instructor
FROM Courses c
JOIN Users u ON u.user_id = c.instructor_id
WHERE c.is_active = TRUE;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| course_code | title | semester | credits | max_students | instructor |
|---|---|---|---|---|---|
| CS301 | Database Management Systems | Spring 2025 | 4 | 5 | Dr. Priya Sharma |
| CS401 | Machine Learning | Spring 2025 | 4 | 5 | Dr. Priya Sharma |
| MATH201 | Discrete Mathematics | Spring 2025 | 3 | 5 | Prof. Arjun Mehta |
| CS302 | Operating Systems | Spring 2025 | 4 | 5 | Dr. Sunita Rao |

Products with Category JOIN — active courses with instructor names

## 6.3 Complex Query Outputs

```sql
SELECT
    s.submission_id,
    u.full_name AS student_name,
    s.marks_obtained,
    s.grade_letter,
    g.full_name AS graded_by
FROM Submissions s
JOIN Users u ON u.user_id = s.student_id
LEFT JOIN Users g ON g.user_id = s.graded_by
WHERE s.assignment_id = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| submission_id | student_name | marks_obtained | grade_letter | graded_by |
|---|---|---|---|---|
| 1 | Aditya Verma | 22.00 | A | Dr. Priya Sharma |
| 2 | Meera Nair | 20.00 | A | Dr. Priya Sharma |
| 3 | Rohan Singh | 18.00 | B | Ravi Kumar |
| 4 | Kavya Patel | 24.00 | A+ | Ravi Kumar |
| 5 | Siddharth Jain | 15.00 | C | Dr. Priya Sharma |

All submissions for Assignment 1 — marks, grade letters, grader names

```sql
SELECT
    p.poll_id,
    p.question,
    po.option_text,
    COUNT(pr.response_id) AS votes,
    ROUND(COUNT(pr.response_id) * 100.0 /
          SUM(COUNT(pr.response_id)) OVER (PARTITION BY p.poll_id), 1) AS pct
FROM Polls p
JOIN Poll_Options po ON po.poll_id = p.poll_id
LEFT JOIN Poll_Responses pr ON pr.option_id = po.option_id
GROUP BY p.poll_id, p.question, po.option_id, po.option_text;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| poll_id | question | option_text | votes | pct |
|---|---|---|---|---|
| 1 | Which topic do you want to cover next? | Transactions & ACID | 1 | 20.0 |
| 1 | Which topic do you want to cover next? | Indexing & Performance Tuning | 2 | 40.0 |
| 1 | Which topic do you want to cover next? | NoSQL Databases | 1 | 20.0 |
| 1 | Which topic do you want to cover next? | Query Optimization | 1 | 20.0 |
| 2 | Rate the difficulty of Assignment 1 (1=Easy): | 1 - Very Easy | 0 | 0.0 |
| 2 | Rate the difficulty of Assignment 1 (1=Easy): | 2 - Easy | 2 | 40.0 |
| 2 | Rate the difficulty of Assignment 1 (1=Easy): | 3 - Moderate | 2 | 40.0 |
| 2 | Rate the difficulty of Assignment 1 (1=Easy): | 4 - Hard | 1 | 20.0 |
| 3 | Should we have a guest lecture on Deep Learni... | Yes, definitely! | 2 | 66.7 |
| 3 | Should we have a guest lecture on Deep Learni... | No, focus on core syllabus | 1 | 33.3 |

Poll results query — vote counts and window-function percentages

```
2        c.course_code,
3        u.full_name AS top_student,
4        e.final_grade
5    FROM Enrollments e
6    JOIN Courses c ON c.course_id = e.course_id
7    JOIN Users u ON u.user_id = e.student_id
8    WHERE e.final_grade = (
9        SELECT MAX(e2.final_grade)
10       FROM Enrollments e2
11       WHERE e2.course_id = e.course_id
12   );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| course_code | top_student | final_grade |
|---|---|---|
| CS301 | Kavya Patel | 96.00 |
| CS401 | Aditya Verma | 90.00 |
| MATH201 | Meera Nair | 93.33 |

Top-scoring student per course — correlated subquery result

## 6.4 Stored Procedure Results

```
CALL sp_register_user(
 'Test Student',
 'test@student.cms.edu',
 SHA2('pass123',256),
 'student',
 '9999999999',
 @uid,
 @msg
);

SELECT @uid, @msg;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| @uid | @msg |
|---|---|
| 15 | User registered successfully with ID 15 |

CALL sp_register_user(…) — new user_id returned

```
CALL sp_grade_submission(
 6,
 2,
 47.0,
 'Good work',
 @msg
);

SELECT @msg;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| @msg |
|---|
| Submission 6 graded: 47.00 marks. |

*CALL sp_grade_submission(…) — grade saved, trigger chain fired*

## 6.5 Trigger Verification

```
1 •  INSERT INTO Enrollments (student_id, course_id)
2    VALUES (7, 1);
```

```
124  13:48:50  INSERT INTO Enrollments (student_id, course_id) VALUES (7, 1)   Error Code: 1644. Enrollment rejected: course has reached maximum capacity.
```

*Enrollment before and after capacity limit — SIGNAL raised on 6th insert*

```
1 •  INSERT INTO Submissions
2    (assignment_id, student_id, content_text)
3    VALUES (2, 7, 'Late submission test');
```

```
127  13:51:53  INSERT INTO Submissions (assignment_id, student_id, content_text) VALUES (2, 7, 'Late submission test')   Error Code: 1644. Submission rejected: deadline has passed and late submissions are not allowed.   0.015 sec
```

*Submission BEFORE INSERT trigger — deadline rejection with SQLSTATE 45000*

```
4    WHERE submission_id = 1;
5
6 •  SELECT final_grade
7    FROM Enrollments
8    WHERE student_id = 7;
```

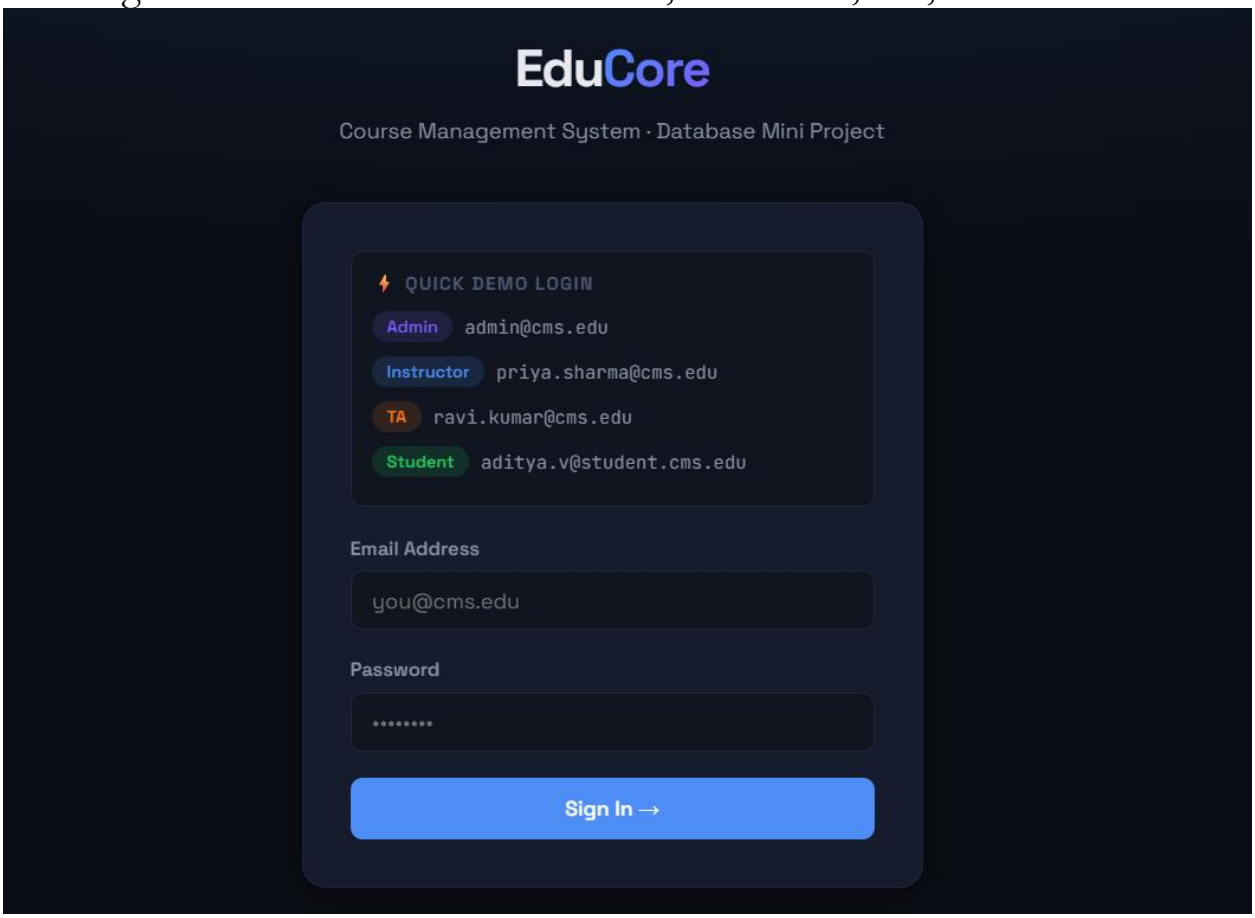
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| final_grade |
|---|
| 92.00 |
| 94.00 |
| NULL |

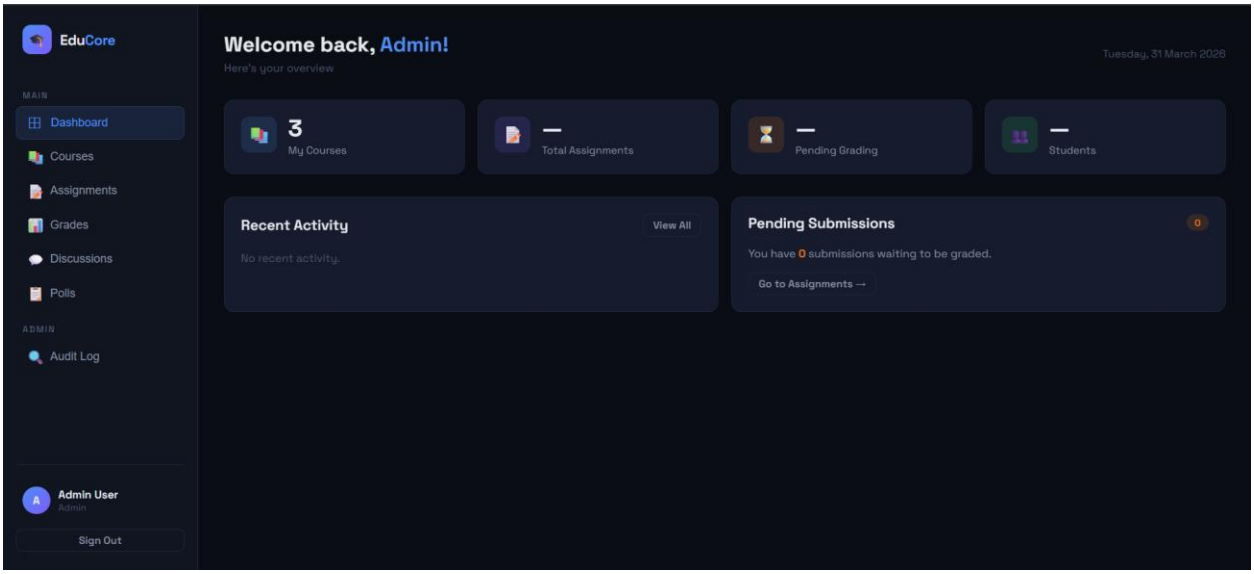
*Enrollments.final_grade auto-updated after grading — Trigger 7 fired*

### 6.6 Web Application Snapshots

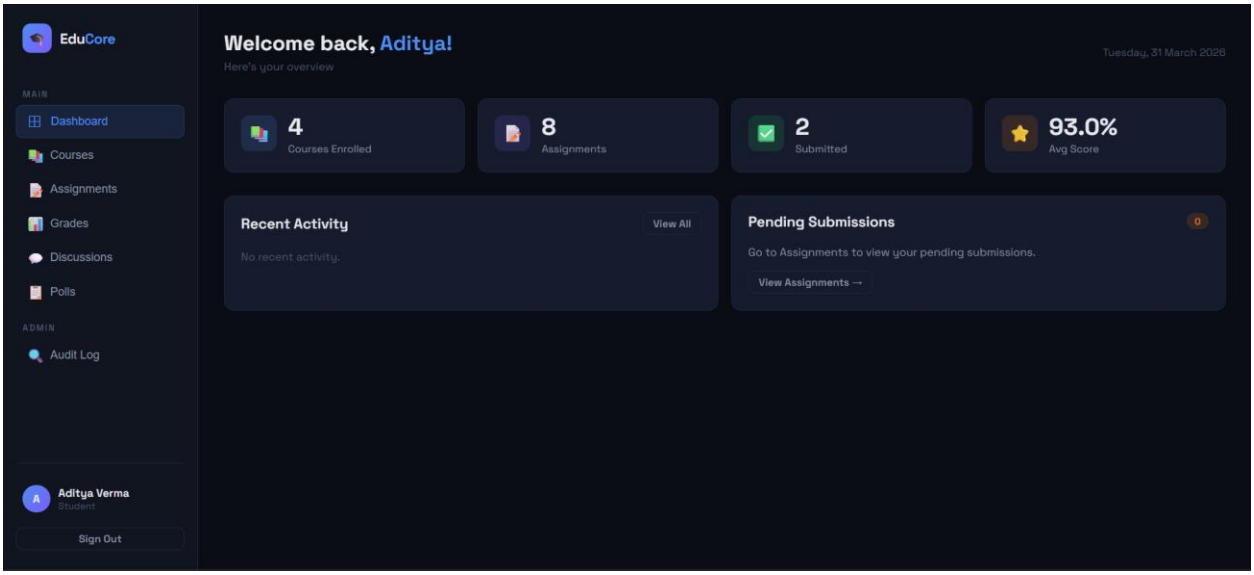
Login Page - Application entry point with Quick Demo Login panel showing clickable credentials for Admin, Instructor, TA, and Student roles.



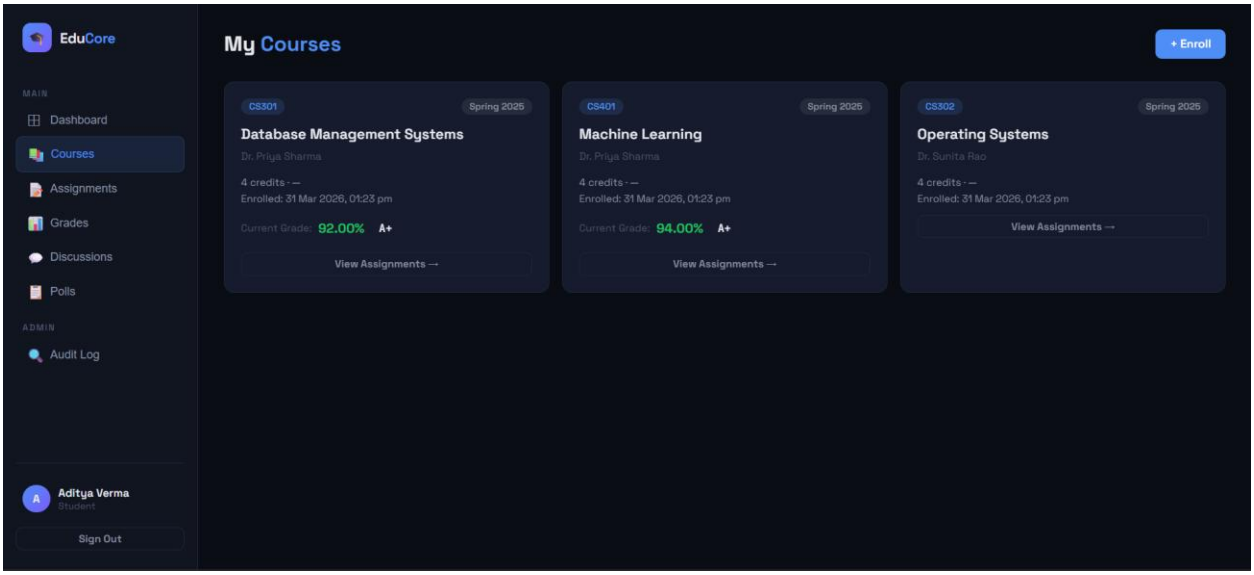

Dashboard (Admin View) - Stat cards for Total Users, Total Courses, Total Enrollments, and Pending Grading count. Recent activity panel showing last 5 audit log entries with action badges.



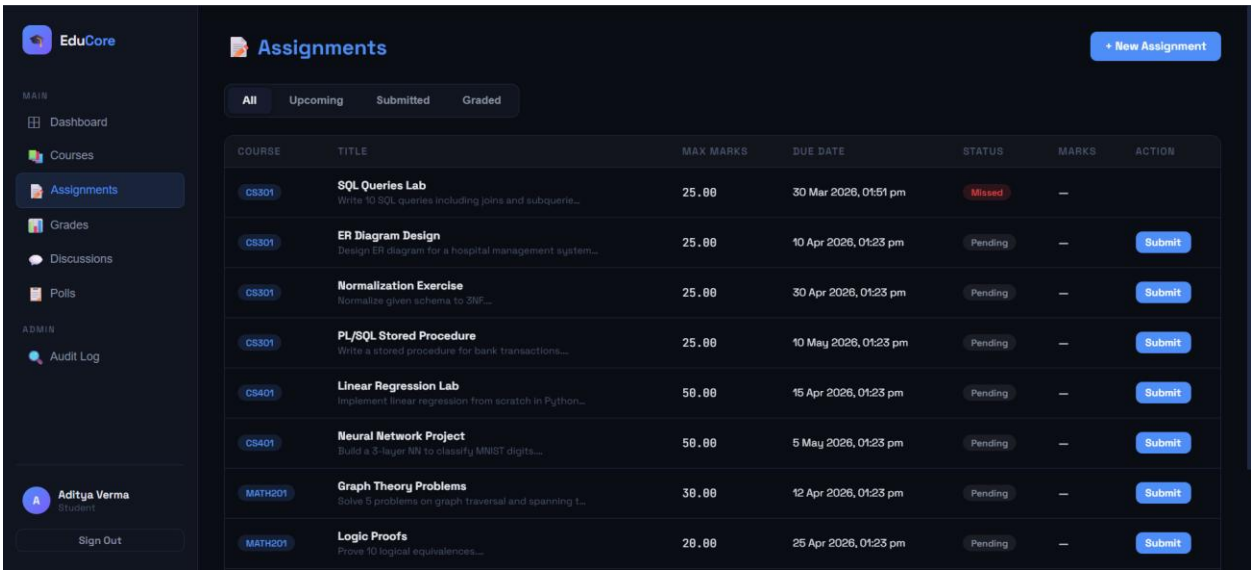

Dashboard (Student View) - Stat cards for Courses Enrolled, Total Assignments, Submitted count, and Average Score percentage.

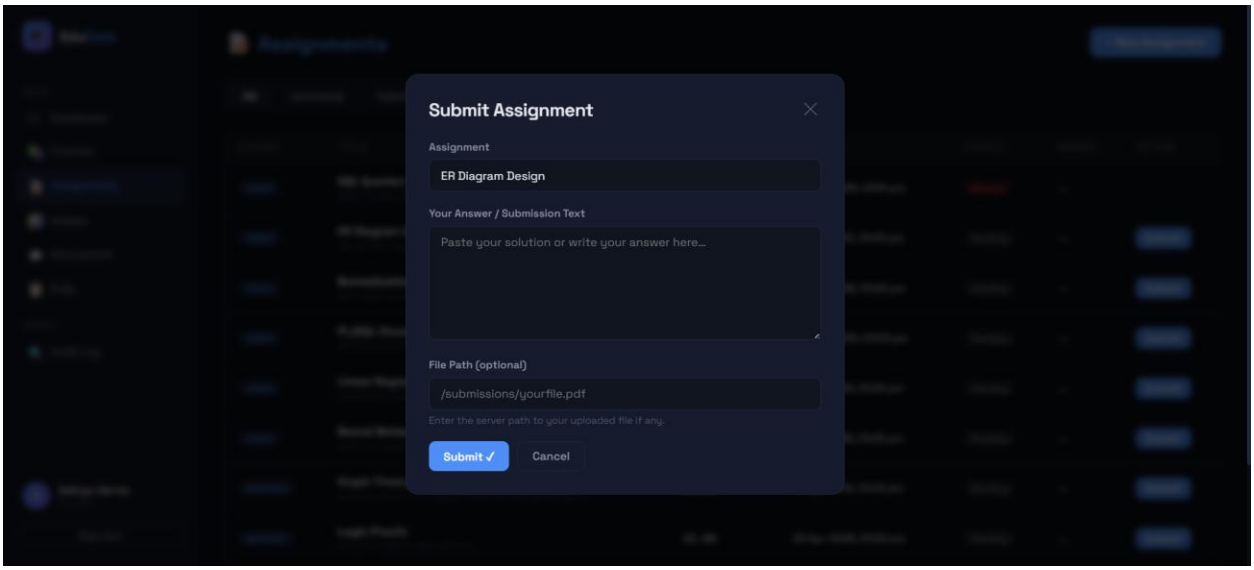



Courses Page - Responsive grid of course cards with enrollment progress bars, fill percentages, credits, and department. Includes Enroll modal for students.

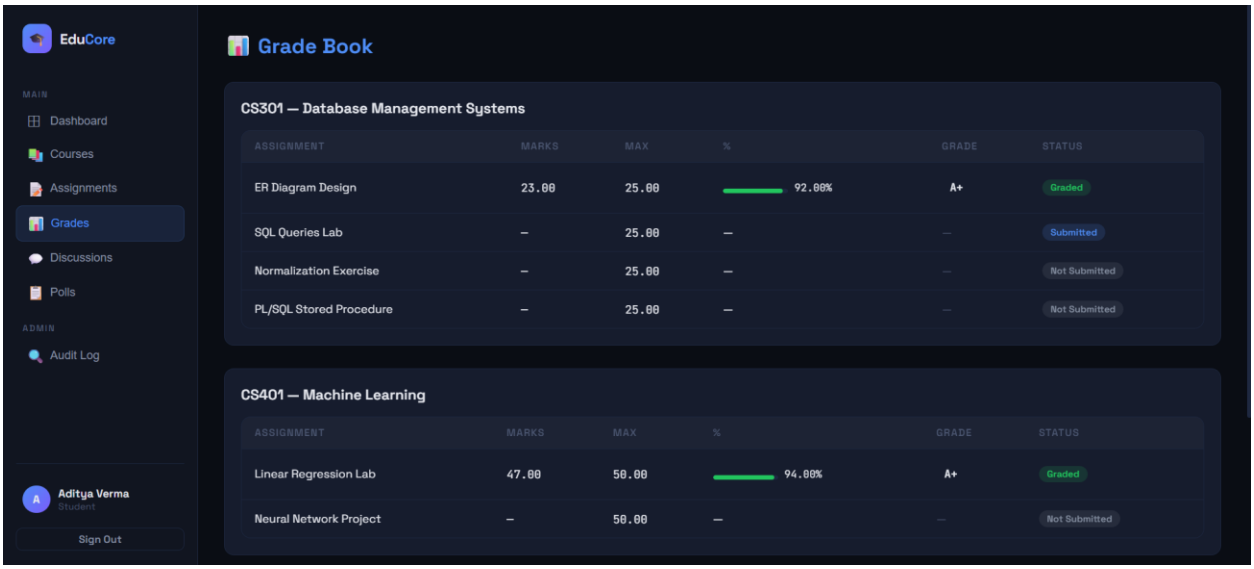



Assignments Page — Tabular view with tabs (All / Upcoming / Submitted / Graded). Status badges, grade pills, and role-specific action buttons (Submit for students, Submissions for instructors).

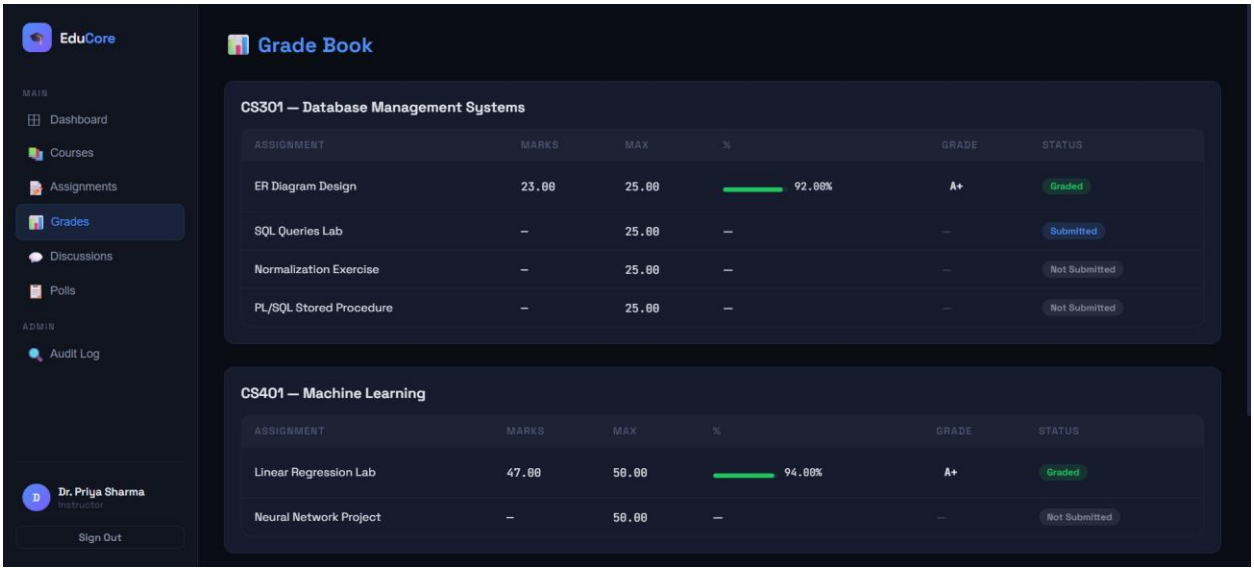




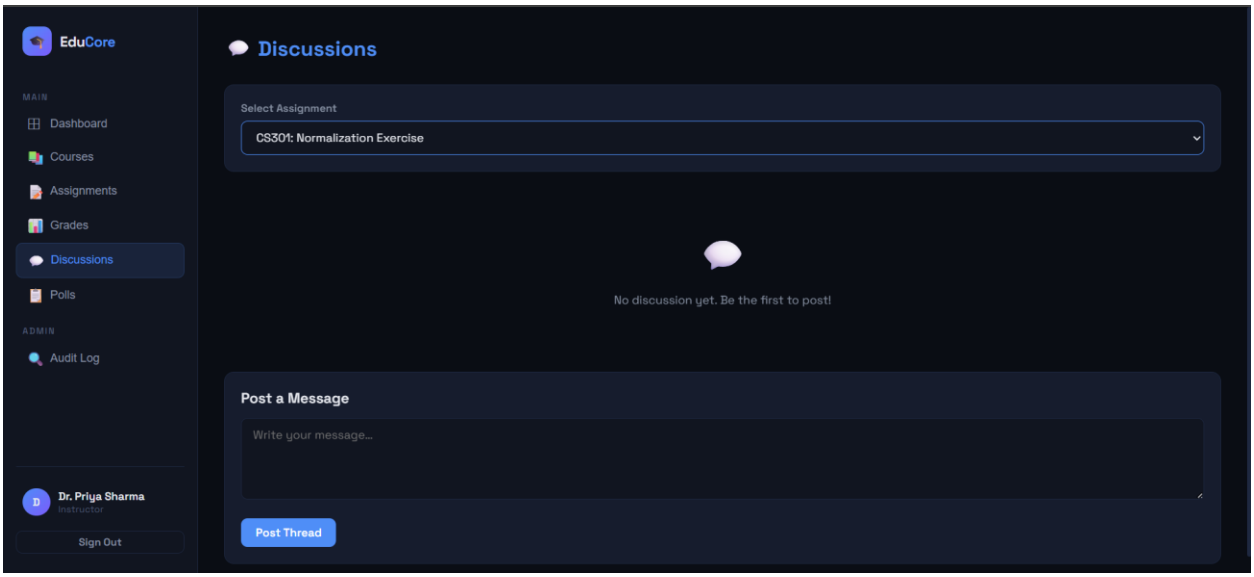

Grades Page (Student View) — Course-grouped grade tables with per-assignment marks, percentage progress bars (colour-coded by score), and grade letter pills.



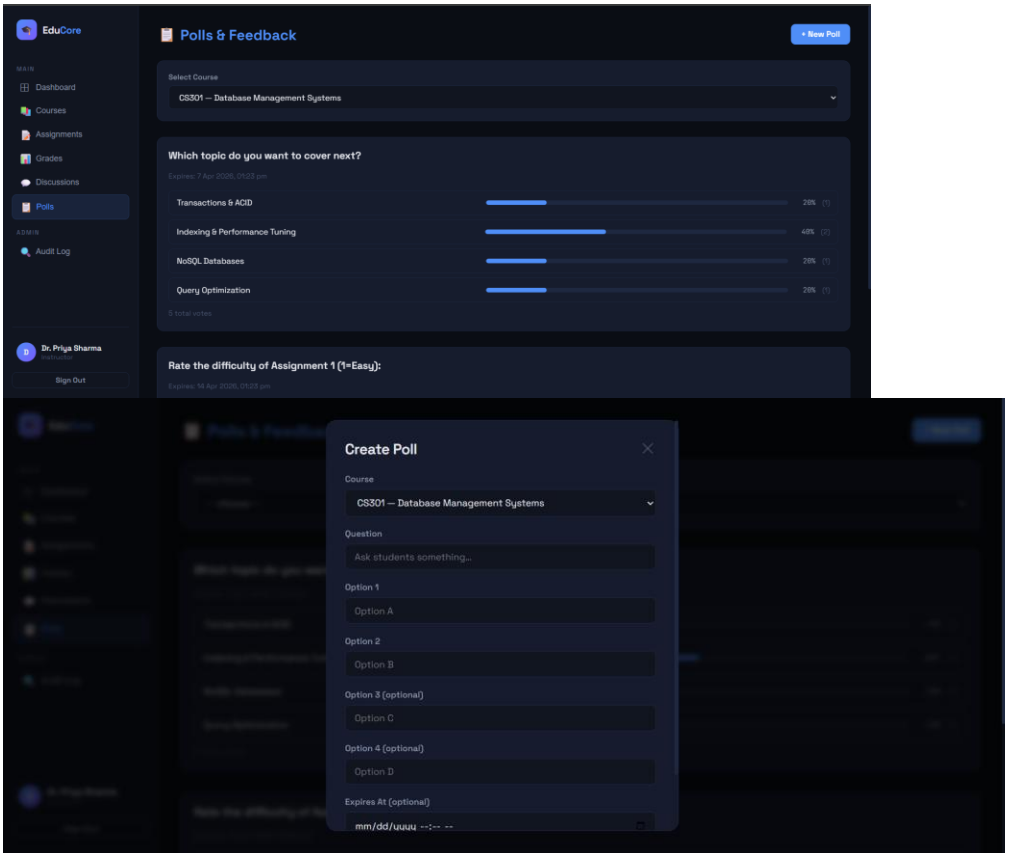
Grades Page (Instructor View) — Class-wide grading progress across all enrolled students and assignments.



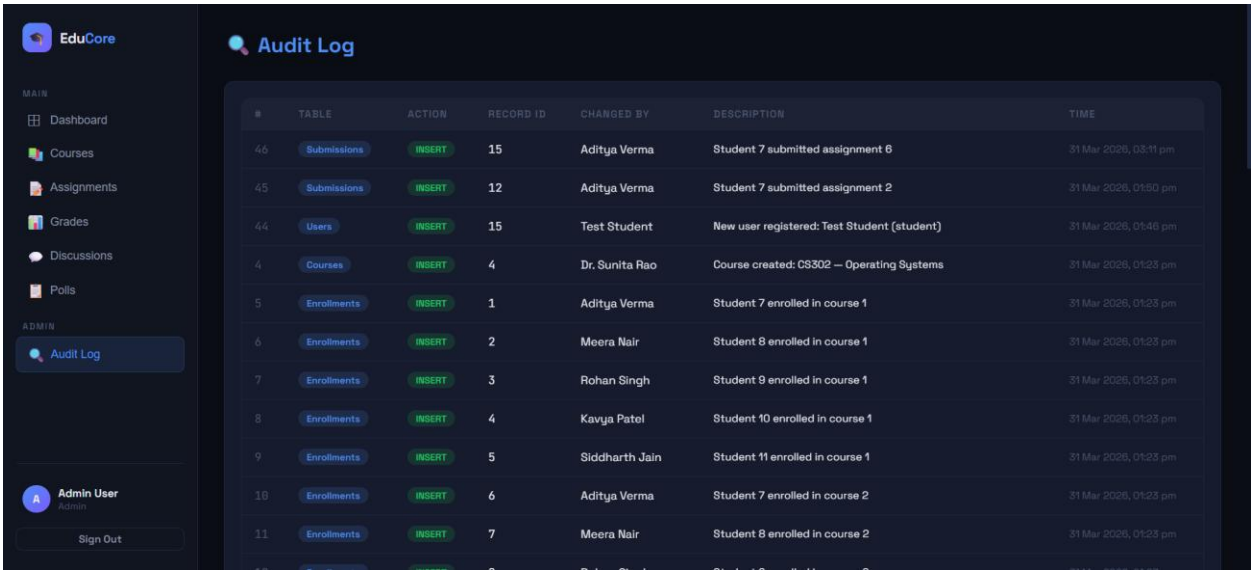

Discussion Page — Assignment selector, pinned thread banners, nested reply sections with author role badges, and a new thread posting area.



Polls Page — Course selector, poll cards with vote distribution bars and percentages, vote submission for students, and poll creation modal for instructors.




Audit Log Page (Admin only) — Full-width table with Log ID, Table Name, Action badge, Record ID, Changed By, Description, and Timestamp columns.




# CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK

## 7.1 Conclusion

The Role-Based Course Management and Assessment System successfully demonstrates the design and implementation of a fully normalized relational database for managing academic operations. The project achieved all stated objectives:

- A complete relational schema was designed and implemented in MySQL with 13 tables, all in 3NF.
- All entities were created with appropriate constraints ensuring data integrity at the database level.

- Basic and complex SQL queries were implemented and tested, covering multi-table JOINs, aggregations, subqueries, and window functions.
- Ten stored procedures encapsulate all core business logic, providing a clean API between the application and the database.
- Ten triggers enforce business rules late submission detection, grade letter computation, final grade recalculation, enrollment capacity, and comprehensive audit logging entirely at the database level without application-level code for each action.
- Eight views provide pre-built reporting and analytics for all user roles.
- A role-adaptive single-page web application demonstrates live database connectivity using Flask and a MySQL connection pool, with JWT-based stateless authentication.
-

The project effectively integrates theoretical DBMS concepts with practical full-stack implementation, serving as a comprehensive demonstration of relational database systems principles in a real-world academic context.

## 7.2 Limitations

- Password compatibility: Seeded sample data uses SHA-256 hashes (via MySQL's SHA2()) for convenience; a legacy fallback is included in auth.py, but production deployments should migrate all users to bcrypt.
- Concurrent grading: The system does not use pessimistic locking on submission rows; simultaneous grading of the same submission by two instructors could result in a race condition without additional row-level locking.
- File storage: The file_path column stores a server file path string only; actual file upload and storage infrastructure is outside the scope of this project.

- Reporting: The system provides tabular data views; graphical trend charts and exportable PDF grade reports are not implemented in the current UI.
- Notification system: No email or in-app notification is triggered when a submission is graded or a deadline is approaching.

### 7.3 Future Work

- Implement real file upload support using a cloud storage backend (e.g., AWS S3 or local MinIO) and store object keys rather than file paths.
- Add WebSocket-based real-time notifications for grade releases and approaching deadlines.
- Extend the admin dashboard with graphical analytics (grade distribution histograms, enrollment trend charts) using a charting library.
- Introduce a plagiarism detection module that compares content_text submissions within the same assignment using similarity scoring.
- Migrate to a production-grade deployment using Gunicorn + Nginx with HTTPS and environment-specific secret management via a vault.
- Implement a mobile-responsive Progressive Web App (PWA) version with offline caching for reading course content and grades.

## CHAPTER 8: REFERENCES


1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.
2. Ramakrishnan, R., & Gehrke, J. (2002). *Database Management Systems* (3rd ed.). McGraw-Hill.
3. MySQL 8.0 Reference Manual. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/
4. Flask Documentation (Version 3.x). Pallets Projects. https://flask.palletsprojects.com/
5. PyJWT Documentation. https://pyjwt.readthedocs.io/

6. bcrypt — Password Hashing Library. https://pypi.org/project/bcrypt/
7. MySQL Connector/Python Documentation. Oracle Corporation. https://dev.mysql.com/doc/connector-python/en/
8. JSON Web Token (JWT) Standard — RFC 7519. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc7519
9. W3Schools SQL Tutorial. https://www.w3schools.com/sql/
10. MDN Web Docs — Fetch API. Mozilla Foundation. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API